ShopEase E-Commerce — Mobile App Build Structure

Oct 10, 2026 · @Vannet

Overview

ShopEase is a mobile e-commerce app whose Figma file has two pages, Page 1 and Page 2, with the Home screen drawn as an iPhone 17 Pro Max frame (node 6:5338). Figma's API call limit on the Starter plan was reached after the first lookup, so the other screens, colors, fonts and components are not read yet. Sections below separate what is confirmed from what is proposed.

The target is Android phones. The Figma frames are drawn for an iPhone 17 Pro Max, so spacing, status and navigation bars, and back navigation need adapting to Android conventions, and layouts should be checked at common phone widths of roughly 360–412 dp rather than one fixed size.

Screens and navigation

Only the Home screen is confirmed from the file; Profile and its four child screens come from your earlier notes on this project.

Screen	Figma node	Status
Home	6:5338	Confirmed in file
Profile	86:4511	Named by you
Payment Methods	—	Planned, linked from Profile
Settings	—	Planned, linked from Profile
Help	—	Planned, linked from Profile
Edit user info	—	Planned, linked from Profile

Still unread: everything else on Page 1 and Page 2 (product list, product detail, cart, checkout, orders or similar). Those rows get added once the file can be read.

Design system

Colors, typography, spacing and reusable components have not been extracted yet, so no values are listed here rather than guessed. Once Figma access resumes:

- [ ] Pull color, type and spacing tokens from the Home screen and Profile screen
- [ ] List repeated components (buttons, product card, tab bar, inputs, list rows) with their variants
- [ ] Export icons and images as assets
- [ ] Map tokens to a theme file in code

Proposed app structure

This is a standard e-commerce layout, not read from the design. It assumes Flutter (Dart) building for Android and should be adjusted to match the real screens.

```
shopease/
├─ android/                 # Android host app: AndroidManifest,
build.gradle.kts, signing
├─ lib/
|  ├─ main.dart            # runApp inside ProviderScope
|  ├─ app/                 # app widget, go_router routes, bottom-nav shell
|  ├─ core/
|  |  ├─ theme/            # app_colors, app_text_styles, app_theme from
Figma tokens
|  |  ├─ network/          # Dio client, interceptors, error mapping
|  |  ├─ storage/          # local database, preferences, secure storage
|  |  └─ widgets/          # buttons, product card, inputs, list rows
|  ├─ features/            # each feature: data / domain / presentation
|  |  ├─ home/
|  |  ├─ search/
|  |  ├─ product/
|  |  ├─ cart/
|  |  ├─ checkout/         # address, payment, review
|  |  └─ profile/          # profile, payment_methods, settings, help,
edit_info
|  └─ l10n/                # ARB string files
├─ assets/                 # icons, images, fonts
├─ test/                   # unit and widget tests
├─ integration_test/       # end-to-end flows
└─ pubspec.yaml
```

The profile feature folder matches the four screens you planned to link from the Profile screen; the other features follow the usual shopping flow and should be renamed or dropped to match the design. Inside each feature, `presentation` holds widgets and controllers, `domain` holds models and use cases, and `data` holds the API and repository code.

Flutter tech stack (proposed)

These are common choices for a Flutter shopping app; swap any that clash with your team's skills.

Area	Proposed choice	Why
Framework	Flutter, latest stable, Dart 3	One codebase, Android first
UI	Material 3 widgets	Native Android look; maps to Figma components
State management	Riverpod	Testable, no BuildContext needed for logic
Navigation	go_router with a ShellRoute	Bottom tabs plus nested profile and checkout flows; handles the Android back button
Networking	Dio	Interceptors for auth tokens and errors
Models	freezed + json_serializable	Immutable models and generated JSON parsing
Images	cached_network_image	Product image caching
Local storage	drift (cart, cache) + shared_preferences (settings)	Offline cart and saved preferences
Secure storage	flutter_secure_storage	Auth tokens on the Android Keystore
Payments	Google Pay via the pay package, or your provider's Flutter SDK	Provider still to be decided
Strings	flutter_localizations + ARB files	Language setting on the Settings screen

Area	Proposed choice	Why
App icon and splash	flutter_launcher_icons, flutter_native_splash	Adaptive icon and splash for Android
Testing	flutter_test, integration_test	Widget tests per screen plus end-to-end flows
Android setup	Own applicationId, Flutter's default minSdk (raise only if a plugin needs it), INTERNET permission	Assumed defaults
Release	flutter build appbundle, signed with an upload key	Google Play requires an AAB

Screen specs (proposed)

Each screen gets a page widget, a Riverpod controller and a go_router route. Contents below are typical for a shopping app and must be checked against the Figma frames.

Screen	Route	Main widgets	Data it needs
Home	/home	Search bar, banner carousel, category chips, product rows	Categories, featured products
Search and listing	/search	Results grid, filter sheet, sort menu	Products by query
Product detail	/product/:id	Image pager, price, option picker, add-to-cart button	One product, reviews
Cart	/cart	Line-item list with quantity steppers, total, checkout button	Cart items, prices
Checkout	/checkout	Address picker, payment choice, order summary	Addresses, payment methods
Profile	/profile	User header, list tiles linking to account screens	User profile
Payment Methods	/profile/payments	Saved method list, add-method form	Payment methods

Screen	Route	Main widgets	Data it needs
Settings	/profile/settings	Switches for notifications, language picker, sign-out	User preferences
Help	/profile/help	FAQ expansion tiles, contact support button	Static or remote content
Edit user info	/profile/edit	Form fields for name, email, phone, photo picker	User profile

Data model (proposed)

Entity	Key fields
User	id, name, email, phone, avatar
Product	id, title, description, price, images, category, stock
CartItem	productId, quantity, selected options
Order	id, items, total, status, address, payment method, created date
Address	id, label, street, city, postal code
PaymentMethod	id, type, last four digits, expiry (no full card numbers stored in the app)

Build plan and open questions

1. Finish reading the Figma file: all screens on both pages, tokens and components.
2. Run `flutter create`, set the Android applicationId, then add the theme, go_router shell and bottom navigation with the Home and Profile screens.
3. Build shared widgets (buttons, product card, inputs), then the remaining screens in the order users meet them, using mock repositories first.
4. Replace mocks with the real API through Dio, add the local cart database, and connect checkout and account screens.

5. Add widget and integration tests, check layouts at 360–412 dp widths on emulators and real phones, then build a signed release bundle (`flutter build appbundle`) for Google Play.

Open questions:

- [ ] Is Riverpod acceptable for state management, or does your team prefer Bloc or Provider?
- [ ] Is there a backend or API yet, or should screens use mock data first?
- [ ] Which payment provider will Payment Methods connect to?
- [ ] Is Page 2 a second flow, a component library or a draft?